

WEEK5-SKILL

Task1:

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
UW PICO 5.09 File: navigation.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <stdlib.h>

int main()
{
    char command[100];
    char path[PATH_MAX];
    char current[PATH_MAX];
    char previous[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    while (1)
    {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "cd") == 0)
        {
            strcpy(previous, current);
            if (chdir(getenv("HOME")) == 0)
            {
                getcwd(current, sizeof(current));
            }
            else
            {
                perror("cd");
            }
            continue;
        }

        if (strncmp(command, "cd ", 3) == 0)
        {
            strcpy(path, command + 3);

            if (strcmp(path, "-") == 0)
            {
                char temp[PATH_MAX];

                if (chdir(previous) == 0)
                {
                    strcpy(temp, current);
                    strcpy(current, previous);
                    strcpy(previous, temp);
                }
                else
            }
        }
    }
}
```

Get Help
Exit
WriteOut
Justify
Read File
Where is
Prev Pg
Next Pg
Cut Text
UnCut Text
Cur Pos
To Spell

```
UW PICO 5.09 File: navigation.c

strcpy(previous, current);
if (chdir(getenv("HOME")) == 0)
{
    getcwd(current, sizeof(current));
}
else
{
    perror("cd");
}
continue;
}
if (strncmp(command, "cd ", 3) == 0)
{
    strcpy(path, command + 3);
    if (strcmp(path, "-") == 0)
    {
        char temp[PATH_MAX];
        if (chdir(previous) == 0)
        {
            strcpy(temp, current);
            strcpy(current, previous);
            strcpy(previous, temp);
        }
        else
        {
            perror("cd");
        }
    }
    else
    {
        strcpy(previous, current);
        if (chdir(path) == 0)
        {
            getcwd(current, sizeof(current));
        }
        else
        {
            perror("cd");
            strcpy(previous, current);
        }
    }
    continue;
}
if (strcmp(command, "pwd") == 0)
{
    printf("%s\n", current);
    continue;
}
printf("Invalid command\n");
}

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell
```

```
UW PICO 5.09 File: navigation.c

        perror("cd");
    }
    else
    {
        strcpy(previous, current);
        if (chdir(path) == 0)
        {
            getcwd(current, sizeof(current));
        }
        else
        {
            perror("cd");
            strcpy(previous, current);
        }
    }
    continue;
}
if (strcmp(command, "pwd") == 0)
{
    printf("%s\n", current);
    continue;
}
printf("Invalid command\n");
}
return 0;
}

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week5skill % nano navigation.c
vishalkarnati@VISHALS-MacBook-Air week5skill % gcc navigation.c -o navigation
vishalkarnati@VISHALS-MacBook-Air week5skill % ./navigation
myshell> pwd
/Users/vishalkarnati/Desktop/os/week5skill
myshell> cd ..
myshell> pwd
invalid command
myshell> pwd
/Users/vishalkarnati/Desktop/os
myshell> cd week5skill
myshell> pwd
/Users/vishalkarnati/Desktop/os/week5skill
myshell> cd ..
myshell> cd -
myshell> cd abcxyz
cd: No such file or directory
myshell> ^C
vishalkarnati@VISHALS-MacBook-Air week5skill %
```

Task2:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
UW PICO 5.09 File: shell.c Modified
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <limits.h>

void builtin_pwd(char **args)
{
    char path[PATH_MAX];
    getcwd(path, sizeof(path));
    printf("%s\n", path);
}

void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        chdir(getenv("HOME"));
        return;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
    }
}

void builtin_exit(char **args)
{
    exit(0);
}

struct builtin
{
    char *name;
    void (*function)(char **);
};

struct builtin builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"exit", builtin_exit}
};

int main()
{
    char command[100];
    char *args[10];

    int builtin_count = 3;

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);
    }
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: shell.c Modified
{
    exit(0);
}

struct builtin
{
    char *name;
    void (*function)(char **);
};

struct builtin builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"exit", builtin_exit}
};

int main()
{
    char command[100];
    char *args[10];

    int builtin_count = 3;

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);

        command[strlen(command)] = '\0';

        if (strlen(command) == 0)
        {
            continue;
        }

        int count = 0;

        char *token = strtok(command, " ");

        while (token != NULL && count < 9)
        {
            args[count] = token;
            count++;

            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        int found = 0;

        for (int i = 0; i < builtin_count; i++)
        {
            if (strcmp(args[0], builtins[i].name) == 0)
            {
                builtins[i].function(args);
                found = 1;
            }
        }
    }
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: shell.c Modified
command[strlen(command, "\n")] = '\0';

if (strlen(command) == 0)
{
    continue;
}

int count = 0;
char *token = strtok(command, " ");
while (token != NULL && count < 9)
{
    args[count] = token;
    count++;

    token = strtok(NULL, " ");
}

args[count] = NULL;
int found = 0;
for (int i = 0; i < builtin_count; i++)
{
    if (strcmp(args[0], builtins[i].name) == 0)
    {
        builtins[i].function(args);
        found = 1;
        break;
    }
}

if (!found)
{
    printf("%s: command not found\n", args[0]);
}

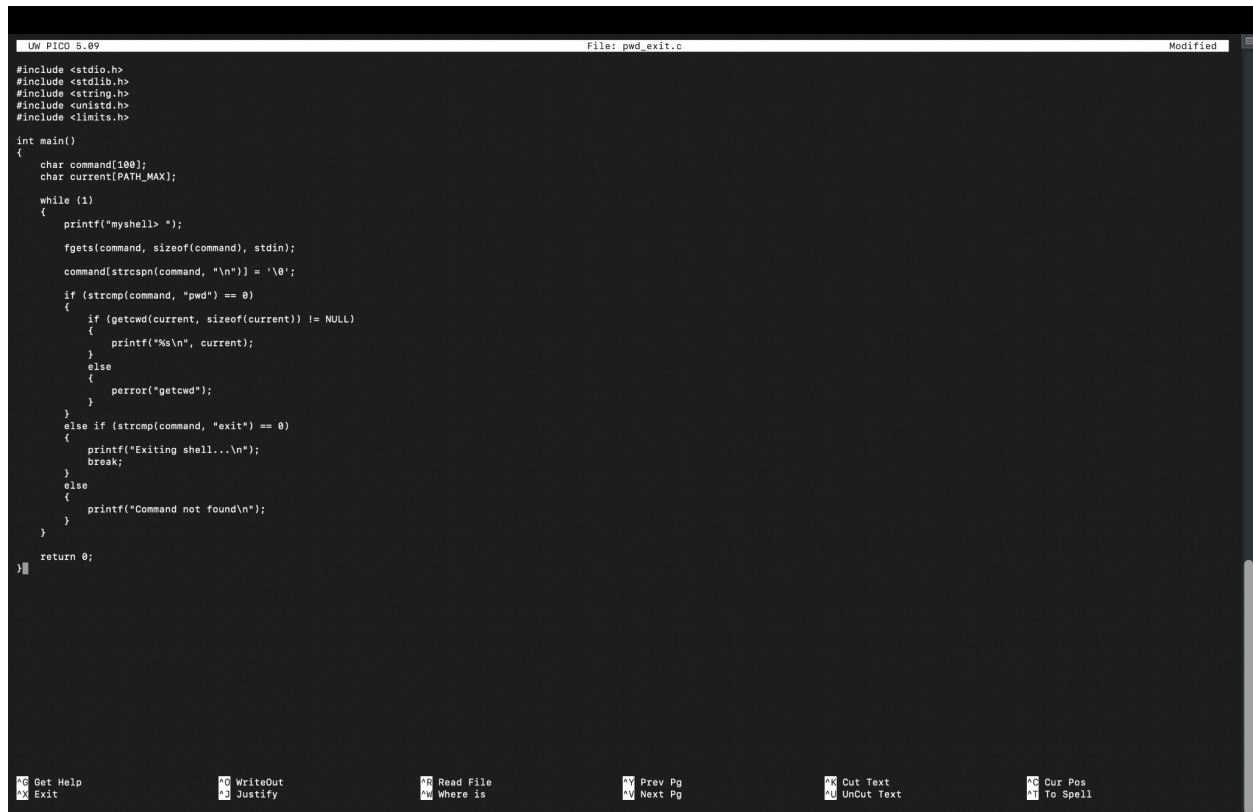
return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week5skill % nano shell.c
vishalkarnati@VISHALS-MacBook-Air week5skill % gcc shell.c -o shell
vishalkarnati@VISHALS-MacBook-Air week5skill % ./shell
myshell> pwd
/Users/vishalkarnati/Desktop/os/week5skill
myshell> cd ..
myshell> pwd
/Users/vishalkarnati/Desktop/os
myshell> hello
hello: command not found
myshell> ^C
vishalkarnati@VISHALS-MacBook-Air week5skill %
```

Task3:

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior



```
UW PIC0 5.09 File: pwd_exit.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char command[100];
    char current[PATH_MAX];

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "pwd") == 0)
        {
            if (getcwd(current, sizeof(current)) != NULL)
            {
                printf("%s\n", current);
            }
            else
            {
                perror("getcwd");
            }
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        else
        {
            printf("Command not found\n");
        }
    }

    return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
vishalkarnati@VISHALS-MacBook-Air week5skill % cd ~/Desktop/os
vishalkarnati@VISHALS-MacBook-Air os % cd week5skill
vishalkarnati@VISHALS-MacBook-Air week5skill % nano pwd_exit.c
vishalkarnati@VISHALS-MacBook-Air week5skill % gcc pwd_exit.c -o pwd_exit
vishalkarnati@VISHALS-MacBook-Air week5skill % ./pwd_exit
myshell> pwd
/Users/vishalkarnati/Desktop/os/week5skill
myshell> hello
myshell> exit
Command not found
myshell> exit
Exiting shell...
vishalkarnati@VISHALS-MacBook-Air week5skill %
```

Task-4:

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
UW PICO 5.09 File: export.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(char *name)
{
    if (name[0] == '\0')
    {
        return 0;
    }

    if (!isalpha(name[0]) && name[0] != '_')
    {
        return 0;
    }

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
        {
            return 0;
        }
    }

    return 1;
}

int main()
{
    char command[200];

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "export ", 7) == 0)
        {
            char *data = command + 7;
            char *equal = strchr(data, '=');

            if (equal == NULL)
            {
                printf("Invalid export syntax\n");
                continue;
            }

            *equal = '\0';

            char *name = data;
            char *value = equal + 1;

            if (!valid_name(name))
            {
                printf("Invalid variable name\n");
                continue;
            }

            if (setenv(name, value, 1) != 0)
            {
                perror("setenv");
            }
            else
            {
                printf("Variable exported successfully\n");
            }
        }
        else if (strcmp(command, "get ", 4) == 0)
        {
            char *name = command + 4;
            char *value = getenv(name);

            if (value != NULL)
            {
                printf("%s\n", value);
            }
            else
            {
                printf("Variable not found\n");
            }
        }
    }
}
```

```
UW PICO 5.09 File: export.c Modified
char command[200];

while (1)
{
    printf("myshell> ");

    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        break;
    }

    if (strcmp(command, "export ", 7) == 0)
    {
        char *data = command + 7;
        char *equal = strchr(data, '=');

        if (equal == NULL)
        {
            printf("Invalid export syntax\n");
            continue;
        }

        *equal = '\0';

        char *name = data;
        char *value = equal + 1;

        if (!valid_name(name))
        {
            printf("Invalid variable name\n");
            continue;
        }

        if (setenv(name, value, 1) != 0)
        {
            perror("setenv");
        }
        else
        {
            printf("Variable exported successfully\n");
        }
    }
    else if (strcmp(command, "get ", 4) == 0)
    {
        char *name = command + 4;
        char *value = getenv(name);

        if (value != NULL)
        {
            printf("%s\n", value);
        }
        else
        {
            printf("Variable not found\n");
        }
    }
}
```

```
UW PICO 5.09 File: export.c Modified
char *value = equal + 1;
if (!valid_name(name))
{
    printf("Invalid variable name\n");
    continue;
}
if (setenv(name, value, 1) != 0)
{
    perror("setenv");
}
else
{
    printf("Variable exported successfully\n");
}
}
else if (strncmp(command, "get ", 4) == 0)
{
    char *name = command + 4;
    char *value = getenv(name);
    if (value != NULL)
    {
        printf("%s\n", value);
    }
    else
    {
        printf("Variable not found\n");
    }
}
else
{
    printf("Command not found\n");
}
}
return 0;
}
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
[vishalkarnati@VISHALS-MacBook-Air week5skill % nano export.c
[vishalkarnati@VISHALS-MacBook-Air week5skill % gcc export.c -o export
[vishalkarnati@VISHALS-MacBook-Air week5skill %
[vishalkarnati@VISHALS-MacBook-Air week5skill % ./export
myshell> export NAME=Lahari
Variable exported successfully
myshell> get NAME
Lahari
myshell> export AGE=18
Variable exported successfully
myshell> get AGE
18
myshell> export NAME=hello
Variable exported successfully
myshell> get NAME
hello
myshell> export 123NAME=hello
Invalid variable name
myshell> export NAME-TEST=hello
Invalid variable name
myshell> exit
[vishalkarnati@VISHALS-MacBook-Air week5skill %
```

